

Correção – Lista de Exercícios

Aula 11 · UML e Modelagem (POO com C#)

Observação: os diagramas usam nomes de atributos e métodos sem acento (como em C#). Cada exercício traz a resolução comentada; as respostas de modelagem aceitam variações, desde que respeitem as regras do enunciado.

Exercício 1 – Conceitos Básicos de UML

Explique com suas palavras: a) O que é UML? b) Qual a importância da modelagem? c) Diferença entre classe e objeto.

a) O que é UML?

UML (*Unified Modeling Language*) é uma linguagem visual padronizada usada para especificar, visualizar e documentar sistemas, principalmente os orientados a objetos. Ela **não é uma linguagem de programação**: é um conjunto de diagramas (classes, casos de uso, sequência, atividades etc.) que representam o sistema de forma que todos da equipe entendam.

b) Importância da modelagem de sistemas

- **Planejar antes de programar**: pensar na estrutura evita retrabalho.
- **Comunicação**: desenvolvedores, analistas e clientes falam a mesma "língua" visual.
- **Detecção precoce de erros**: é bem mais barato corrigir um diagrama do que um sistema pronto.
- **Documentação**: facilita a manutenção e a entrada de novas pessoas no projeto.
- **Base para o código**: cada classe do diagrama vira uma classe em C#.

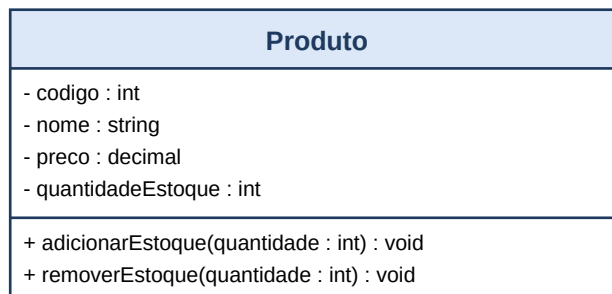
c) Classe x Objeto

Classe é o molde (a definição): descreve quais atributos e métodos existirão. **Objeto** é uma instância concreta criada a partir desse molde, com valores próprios. Exemplo: *Cliente* é a classe; "Maria" e "João" são objetos da classe Cliente.

```
Cliente c1 = new Cliente(); // c1 é um objeto da classe Cliente
Cliente c2 = new Cliente(); // c2 é outro objeto, independente de c1
```

Exercício 2 – Estrutura de Classe UML

Diagrama de classe para *Produto* (código, nome, preço, quantidadeEstoque; adicionarEstoque(), removerEstoque()).



A classe UML tem três compartimentos: **nome** (topo), **atributos** (meio) e **métodos** (base). Formato dos atributos: visibilidade nome : tipo. Formato dos métodos: visibilidade nome(parâmetros) : retorno.

Exercício 3 – Visibilidade em UML

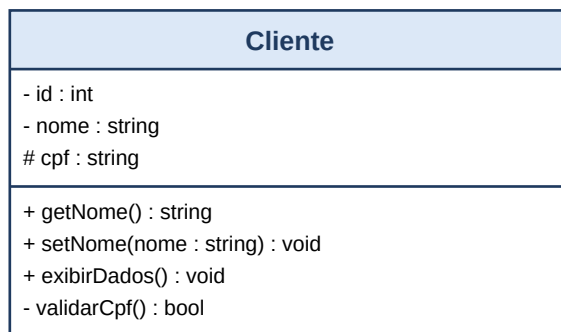
Identifique os símbolos de atributo privado, método público e atributo protegido e crie a classe *Cliente*.

Respostas

- Atributo privado -> - (menos)
- Método público -> + (mais)
- Atributo protegido -> # (jogo da velha)

Símbolo	Visibilidade	Equivalente em C#	Exemplo
+	Público	public	+ getNome() : string
-	Privado	private	- saldo : double
#	Protegido	protected	# cpf : string
~	Pacote	internal	~ codigoInterno : int

Classe *Cliente* com a visibilidade aplicada



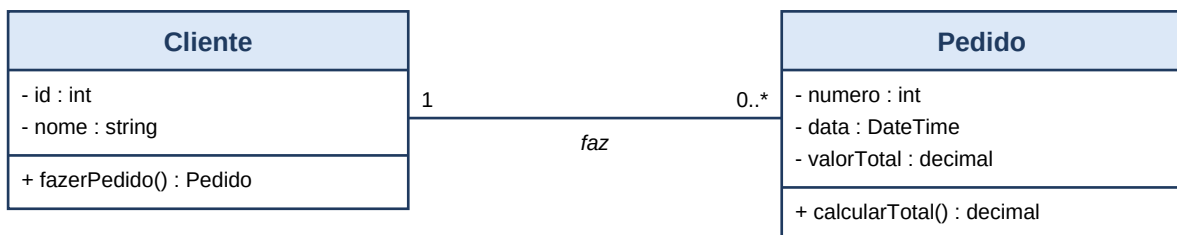
Legenda:

- + público
- privado
- # protegido

Os dados (*id*, *nome*) ficam privados para proteger o *encapsulamento*; o *cpf* é protegido (subclasses podem acessar); os métodos de uso externo são públicos; e a validação é um método privado, de uso interno da classe.

Exercício 4 – Associação

Cliente e *Pedido*: um cliente pode fazer vários pedidos; um pedido pertence a apenas um cliente.



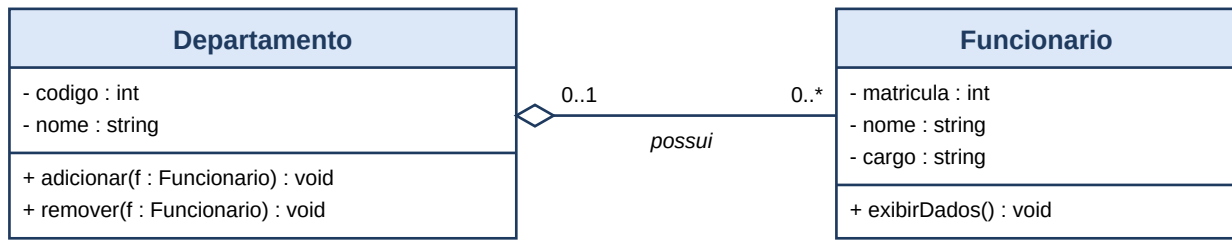
Como ler a multiplicidade

- Lado do **Cliente** = **1**: cada pedido pertence a exatamente um cliente.
- Lado do **Pedido** = **0..***: um cliente pode ter nenhum, um ou vários pedidos.

Associação é o relacionamento mais simples: "um conhece o outro", representado por uma linha contínua.

Exercício 5 – Agregação

Departamento e Funcionário: um departamento possui vários funcionários, que podem existir sem o departamento.



Por que é agregação?

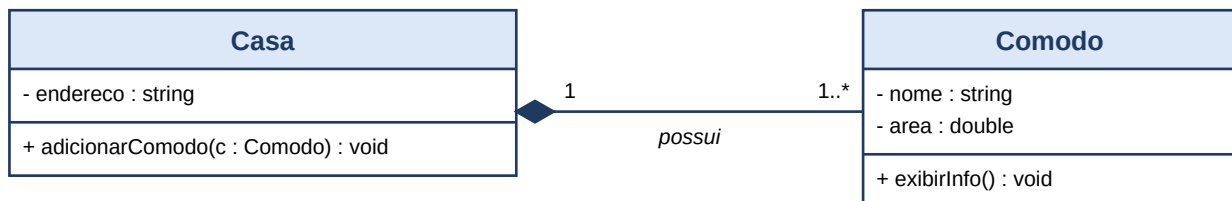
Agregação é uma relação **todo–parte fraca**: o departamento (todo) reúne funcionários (partes), mas as partes têm **ciclo de vida independente**. Se o departamento for extinto, os funcionários continuam existindo (podem ser realocados). Por isso o losango é **vazio** e fica do lado do "todo". Note também o 0..1 no lado do departamento: o funcionário pode estar sem departamento.

```
public class Departamento
{
    public List<Funcionario> Funcionarios { get; } = new List<Funcionario>();

    // O funcionário é criado FORA e apenas recebido: agregação
    public void Adicionar(Funcionario f) => Funcionarios.Add(f);
}
```

Exercício 6 – Composição

Casa e Cômodo: uma casa possui vários cômodos; um cômodo não existe sem a casa.



Por que é composição?

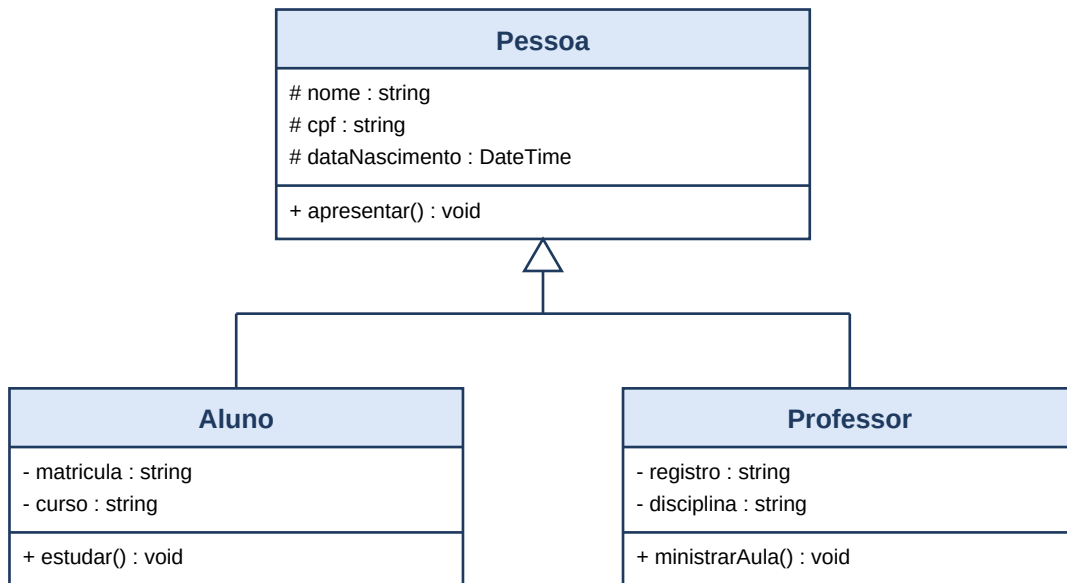
Composição é uma relação **todo–parte forte**: a parte **depende** do todo. Se a casa deixa de existir, seus cômodos também deixam. O losango é **preenchido**, do lado do todo, e a multiplicidade do lado da Casa é exatamente 1 (todo cômodo pertence a uma única casa). Em C#, o todo costuma **criar** as partes internamente:

```
public class Casa
{
    private List<Comodo> comodos = new List<Comodo>();

    // O cômodo nasce DENTRO da casa: composição
    public void AdicionarComodo(string nome, double area)
    {
        comodos.Add(new Comodo(nome, area));
    }
}
```

Exercício 7 – Herança

Classe pai Pessoa; classes filhas Aluno e Professor.



A herança é representada por uma seta com **triângulo vazado** apontando para a classe pai e lê-se "**é um**": Aluno é uma Pessoa; Professor é uma Pessoa. Os atributos comuns ficam em Pessoa (protegidos, para as filhas acessarem) e cada filha acrescenta o que é específico dela.

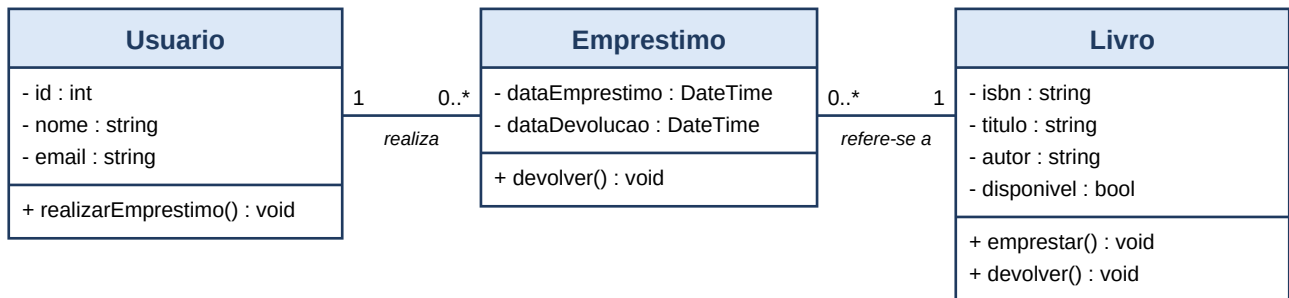
```
public class Pessoa
{
    protected string nome;
    protected string cpf;
    protected DateTime dataNascimento;
    public void Apresentar() { /* ... */ }
}

public class Aluno : Pessoa    // Aluno herda de Pessoa
{
    private string matricula;
    private string curso;
    public void Estudar() { /* ... */ }
}

public class Professor : Pessoa // Professor herda de Pessoa
{
    private string registro;
    private string disciplina;
    public void MinistrarAula() { /* ... */ }
}
```

Exercício 8 – Sistema de Biblioteca

Livro, Usuário e Empréstimo: um usuário realiza vários empréstimos; um empréstimo pertence a um único livro.



- **Usuario 1 — 0..* Emprestimo:** um usuário pode realizar vários empréstimos; cada empréstimo é de um só usuário.
- **Emprestimo 0..* — 1 Livro:** cada empréstimo se refere a um único livro; ao longo do tempo, um mesmo livro pode aparecer em vários empréstimos.

A classe **Emprestimo** faz a "ponte" entre **Usuario** e **Livro** e guarda as informações do próprio empréstimo (datas de empréstimo e devolução).

Exercício 9 – Conversão UML para C#

Transforme o diagrama de ContaBancaria (- saldo : double; + depositar() : void; + sacar() : void) em C#.

```
public class ContaBancaria
{
    private double saldo;           // "-" vira private

    public void Depositar(double valor) // "+" vira public
    {
        if (valor > 0)
            saldo += valor;
    }

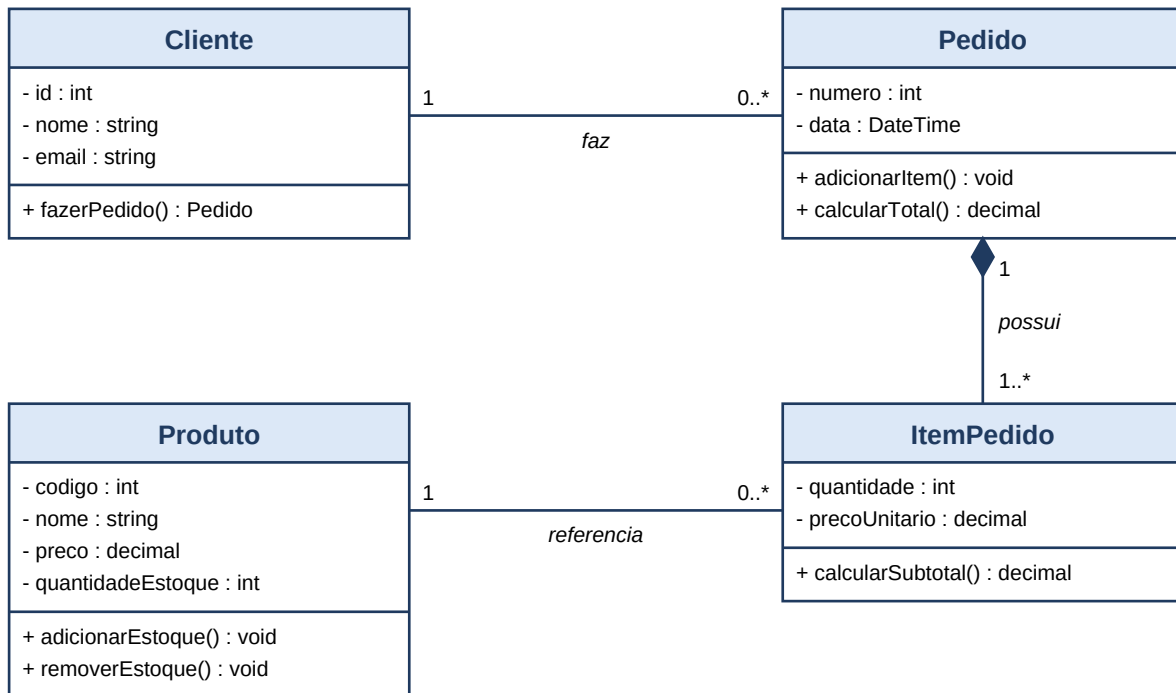
    public void Sacar(double valor)
    {
        if (valor > 0 && valor <= saldo)
            saldo -= valor;
    }
}
```

Pontos importantes

- `- saldo : double` -> `private double saldo;`. O tipo vem **depois** do nome na UML e **antes** do nome em C#.
- `+ depositar() : void` -> método público sem retorno. Na UML a lista de parâmetros pode estar omitida; no código é natural receber o valor como parâmetro.
- Por convenção, métodos em C# usam PascalCase (`Depositar`, `Sacar`).
- As validações garantem que o saldo nunca fique negativo; é o encapsulamento em ação.

Exercício 10 – Modelagem Completa (Sistema de Vendas)

Cliente, Produto, Pedido e ItemPedido: um pedido possui vários itens; cada item referencia um produto; um cliente pode ter vários pedidos.



- **Cliente 1 — 0..* Pedido:** associação (o cliente faz vários pedidos).
- **Pedido (losango cheio) 1..* ItemPedido:** composição, pois um item de pedido não existe sem o pedido ao qual pertence. (Quem preferir pode usar agregação; o importante é justificar.)
- **ItemPedido 0..* — 1 Produto:** cada item referencia um único produto, e um produto pode estar em vários itens.

O valor total do pedido não precisa ser um atributo: ele é **calculado** pelo método `calcularTotal()` somando os subtotais dos itens.

Desafio extra – Implementação em C#

```
using System;
using System.Collections.Generic;

public class Cliente
{
    public int Id { get; }
    public string Nome { get; }
    public string Email { get; }
    public List<Pedido> Pedidos { get; } = new List<Pedido>();

    public Cliente(int id, string nome, string email)
    {
        Id = id;
        Nome = nome;
        Email = email;
    }

    public Pedido FazerPedido(int numero)
    {
        Pedido pedido = new Pedido(numero, this);
        Pedidos.Add(pedido);
        return pedido;
    }
}

public class Produto
{
    public intCodigo { get; }
    public string Nome { get; }
    public decimal Preco { get; }
    public int QuantidadeEstoque { get; private set; }

    public Produto(int codigo, string nome, decimal preco, int quantidadeEstoque)
    {
        Codigo = codigo;
        Nome = nome;
        Preco = preco;
        QuantidadeEstoque = quantidadeEstoque;
    }

    public void AdicionarEstoque(int quantidade)
    {
        if (quantidade > 0)
            QuantidadeEstoque += quantidade;
    }

    public bool RemoverEstoque(int quantidade)
    {
        if (quantidade <= 0 || quantidade > QuantidadeEstoque)
            return false;

        QuantidadeEstoque -= quantidade;
        return true;
    }
}

public class ItemPedido
{
    public Produto Produto { get; }
    public int Quantidade { get; }
    public decimal PrecoUnitario { get; }

    public ItemPedido(Produto produto, int quantidade)
```

```

        {
            Produto = produto;
            Quantidade = quantidade;
            PrecoUnitario = produto.Preco;    // preço "congelado" no momento da compra
        }

        public decimal CalcularSubtotal() => PrecoUnitario * Quantidade;
    }

    public class Pedido
    {
        public int Numero { get; }
        public DateTime Data { get; }
        public Cliente Cliente { get; }

        private readonly List<ItemPedido> itens = new List<ItemPedido>();
        public IReadOnlyList<ItemPedido> Itens => itens;

        public Pedido(int numero, Cliente cliente)
        {
            Numero = numero;
            Data = DateTime.Now;
            Cliente = cliente;
        }

        public void AdicionarItem(Produto produto, int quantidade)
        {
            if (!produto.RemoverEstoque(quantidade))
                throw new InvalidOperationException("Quantidade inválida ou estoque insuficiente.");

            itens.Add(new ItemPedido(produto, quantidade)); // o item nasce dentro do pedido
        }

        public decimal CalcularTotal()
        {
            decimal total = 0;
            foreach (ItemPedido item in itens)
                total += item.CalcularSubtotal();
            return total;
        }
    }
}

```

Exemplo de uso (Program.cs)

```

class Program
{
    static void Main()
    {
        Cliente cliente = new Cliente(1, "Maria", "maria@email.com");
        Produto notebook = new Produto(101, "Notebook", 3500m, 10);
        Produto mouse = new Produto(102, "Mouse", 80m, 50);

        Pedido pedido = cliente.FazerPedido(1);
        pedido.AdicionarItem(notebook, 1);
        pedido.AdicionarItem(mouse, 2);

        Console.WriteLine($"Pedido {pedido.Numero} - Cliente: {pedido.Cliente.Nome}");
        foreach (ItemPedido item in pedido.Itens)
            Console.WriteLine($"{item.Quantidade}x {item.Produto.Nome} = {item.CalcularSubtotal():C}");
        Console.WriteLine($"Total: {pedido.CalcularTotal():C}");
    }
}

```